

---

# Modül 12 — Spring Security ile Güvenlik

Ders Notları

Spring Boot Edu



2026-09-24

## İçindekiler

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>1. Öğrenme Hedefleri</b>                         | <b>2</b>  |
| <b>2. Kavramlar</b>                                 | <b>2</b>  |
| 2.1 Kimlik Doğrulama ve Yetkilendirme . . . . .     | 2         |
| 2.2 Filter Chain . . . . .                          | 2         |
| 2.3 Oturumlar, CSRF ve CORS . . . . .               | 3         |
| <b>3. Adım Adım Örnekler</b>                        | <b>3</b>  |
| 3.1 Tarayıcı Zinciri . . . . .                      | 4         |
| 3.2 Kullanıcılar ve Parolalar . . . . .             | 5         |
| 3.3 API Zinciri . . . . .                           | 6         |
| 3.4 Method Security . . . . .                       | 7         |
| 3.5 JWT Resource Server . . . . .                   | 8         |
| 3.6 Kendi Authorization Server'ımız . . . . .       | 8         |
| 3.7 OAuth2 Login ("GitHub ile giriş yap") . . . . . | 10        |
| 3.8 CORS . . . . .                                  | 10        |
| 3.9 Test . . . . .                                  | 11        |
| <b>4. Sık Yapılan Hatalar ve En İyi Pratikler</b>   | <b>11</b> |
| <b>5. Özet</b>                                      | <b>12</b> |
| <b>6. İleri Okuma</b>                               | <b>12</b> |

## 1. Öğrenme Hedefleri

Bu modülün sonunda şunları yapabileceksiniz:

- Spring Security filter chain'inin her istek hakkında nasıl karar verdiğini açıklamak
- Birden çok SecurityFilterChain kullanmak: API için stateless, tarayıcı için oturum tabanlı
- Kullanıcıları PostgreSQL'de, parolaları tuzlanmış BCrypt hash'leri olarak saklamak
- Form login, HTTP Basic ve bearer token arasında seçim yapmak; CSRF ve CORS'un ne zaman önemli olduğunu bilmek
- Servis metotlarını @PreAuthorize ve @PostAuthorize ile korumak
- Spring Authorization Server ile JWT üretmek ve resource server olarak kabul etmek
- "GitHub ile giriş yap" sunmak (OAuth2 login)
- Tüm bunları spring-security-test ile test etmek

**Ön koşullar:** Modül 03 (REST) ve 06 (PostgreSQL) · **Tahmini süre:** 6 saat · **Docker gerekir**

## 2. Kavramlar

### 2.1 Kimlik Doğrulama ve Yetkilendirme

| Kimlik doğrulama (authentication)                             | Yetkilendirme (authorization)                                           |
|---------------------------------------------------------------|-------------------------------------------------------------------------|
| <b>Kimsin?</b>                                                | <b>Ne yapabilirsin?</b>                                                 |
| Parola, token, sertifika                                      | Roller, scope'lar, sahiplik kuralları                                   |
| <b>401 Unauthorized</b> ile başarısız olur ("önce giriş yap") | <b>403 Forbidden</b> ile başarısız olur ("seni tanıyoruz ama izin yok") |

Kimlik doğrulamanın sonucu, SecurityContext içindeki bir Authentication nesnesidir: bir ad ve ROLE\_ADMIN veya SCOPE\_books.write gibi **yetkilerin** (authority) listesi. hasRole('ADMIN'), hasAuthority('ROLE\_ADMIN') ile aynı kontroldür.

### 2.2 Filter Chain

Spring Security, controller'ınızdan **önce** çalışan bir servlet filtre zinciridir. Her SecurityFilterChain bean'inin bir eşleyicisi (matcher) vardır ve eşleyicisi uyan ilk zincir isteği işler:

```

istek → zincir 1: /oauth2/**, /.well-known/** (Authorization Server)
      → zincir 2: /api/** (stateless: Basic veya JWT)
      → zincir 3: geri kalan her şey (tarayıcı: form login, oturum, CSRF)

```

Bir zincirin içinde filtreler isteğin kimliğini doğrular (form, Basic, bearer token...). Sonuncusu olan `AuthorizationFilter` kuralları kontrol eder.

## 2.3 Oturumlar, CSRF ve CORS

- Tarayıcı bir **oturum** cookie'sini sitenize yapılan her istekle, isteği başka bir site te-tiklese bile, kendiliğinden gönderir. **CSRF** saldırısının kötüye kullandığı budur. Bu yüzden CSRF koruması cookie tabanlı (tarayıcı) uygulamalar için önemlidir ve Spring Security onu varsayılan olarak açar.
- Her istekte açık bir Authorization header'ı bekleyen **stateless** bir API'de kötüye kul-lanılacak bir cookie yoktur.
- **CORS** ise ters yöne ilgilidir: Başka bir origin'den gelen JavaScript'in API'nizin yanıt-larını okuyup okuyamayacağı. Tarayıcı önce bir preflight OPTIONS isteğiyle sorar.

## 3. Adım Adım Örnekler

Docker açıkken uygulamayı başlatın. PostgreSQL kök dizindeki `compose.yaml` dosyasından başlar:

```
./mvnw -pl modules/12-security/lesson spring-boot:run
```

`spring-boot:run` **dev** profilini etkinleştirir. Demo kullanıcıları ve demo client secret'ını yalnızca bu profil yükler:

```

spring:
  flyway:
    locations: classpath:db/migration,classpath:db/demo # + demo users with known
    ↪ passwords
  security:
    oauth2:
      authorizationserver:
        client:
          bookstore-cli:
            registration:
              client-secret: "{noop}dev-only-secret" # plain text: acceptable
    ↪ only on your machine
  bookstore:
    tour:

```

```
enabled: true # the tour uses the demo
↪ credentials
```

Dev profili olmadan demo kullanıcılar yoktur ve BOOKSTORE\_CLI\_SECRET ortam değişkeni client secret'ı (ideal olarak {bcrypt}... biçiminde) içermedikçe uygulama başlamayı reddeder. Deneyin: `./mvnw -pl modules/12-security/lesson spring-boot:run -Dspring-boot.run.profiles=prod` komutu `Could not resolve placeholder 'BOOKSTORE_CLI_SECRET'` hatasıyla durur.

Tur (LessonTour, yalnızca dev profilinde) çalışan sunucuyu gerçek bir client gibi HTTP üzerinden çağırır:

```
== 3.3 Public and protected
GET /api/books (anonymous): [{"isbn":"9780134685991","title":"Effective Java"}, ...]
POST /api/books as ada (CUSTOMER): 403 Forbidden
== 3.6 A token from our Authorization Server
access_token: eyJraWQiOiI3MGE2MWFjMC00YzAxLTRhMWItYjAz...
== 3.5 Calling the API with the JWT
GET /api/me: {"name":"bookstore-
↪ cli","type":"JwtAuthenticationToken","authorities":["FACTOR_BEARER","SCOPE_books.read","SCOPE_book...
```

### ! Uyarı

Bilinen demo parolaları ve {noop} secret'lar yalnızca kendi bilgisayarınızda kabul edilebilir. Bu yüzden dev profilinde ve bir kurulumun asla yüklenmediği ayrı db/demo Flyway konumunda dururlar.

## 3.1 Tarayıcı Zinciri

```
@Bean
@Order(3)
SecurityFilterChain webChain(HttpSecurity http,
↪ ObjectProvider<ClientRegistrationRepository> oauth2Clients)
throws Exception {
    http.authorizeHttpRequests(authorize -> authorize
        .requestMatchers("/", "/error").permitAll()
        .anyRequest().authenticated())
        .formLogin(withDefaults()) // Spring Security generates
        ↪ /login
        .logout(withDefaults()); // CSRF protection stays on
        ↪ (the default)
    if (oauth2Clients.getIfAvailable() != null) { // only if a provider is
        ↪ configured (lesson 3.7)
        http.oauth2Login(withDefaults());
```

```

    }
    return http.build();
}

```

- / herkese açıktır. Geri kalan her şey giriş ister. /account için anonim bir istek /login'e yönlendirilir.
- formLogin() bir giriş sayfası üretir ve POST /login'i işler. Başarılı girişten sonra kullanıcı istediği sayfaya veya /'a gönderilir. Yanlış parola /login?error'a götürür.
- CSRF koruması açık kalır: Tarayıcıdan gelen bir POST, Spring Security'nin formlarına koyduğu gizli CSRF token'ını ister. WebChainTest, token olmadan aynı POST'un **403** aldığını gösterir.

## 3.2 Kullanıcılar ve Parolalar

Kullanıcılar PostgreSQL'de, JdbcUserDetailsManager'ın beklediği tablolarda durur:

```

-- The default schema of JdbcUserDetailsManager (see
↳ org/springframework/security/core/userdetails/jdbc/users.ddl)
CREATE TABLE users (
    username VARCHAR(50) NOT NULL PRIMARY KEY,
    password VARCHAR(500) NOT NULL,           -- "{bcrypt}$2a$10$...": the prefix names
    ↳ the algorithm
    enabled BOOLEAN NOT NULL
);

CREATE TABLE authorities (
    username VARCHAR(50) NOT NULL REFERENCES users (username),
    authority VARCHAR(50) NOT NULL           -- "ROLE_ADMIN" → hasRole('ADMIN')
);

CREATE UNIQUE INDEX ix_auth_username ON authorities (username, authority);

```

```

@Bean
UserDetailsManager users(dataSource) {
    return new JdbcUserDetailsManager(dataSource);    // tables "users" and
    ↳ "authorities" (Flyway V1)
}

@Bean
PasswordEncoder passwordEncoder() {
    return PasswordEncoderFactories.createDelegatingPasswordEncoder(); // bcrypt now,
    ↳ other ids still readable
}

```

- Parolalar {bcrypt}\$2a\$10\$... olarak saklanır. BCrypt rastgele bir **tuz** (salt) ekler (aynı parola her seferinde farklı bir hash verir) ve bilerek yavaştır. Böylece çalınan hash'leri kırmak pahalı olur.
- Önek algoritmanın adıdır. DelegatingPasswordEncoder yeni parolaları BCrypt ile kodlar, başka önekli eski hash'leri de hâlâ kontrol edebilir. Böylece herkesi parola sıfırlamaya zorlamadan algoritmayı değiştirebilirsiniz.
- Form login ve HTTP Basic, ikisi de bu UserDetailsService bean'ini kullanır.
- Şema db/migration'dan (tüm ortamlar), demo kullanıcılar db/demo'dan (yalnızca dev profili) gelir.

### 3.3 API Zinciri

```
@Bean
@Order(2)
SecurityFilterChain apiChain(HttpSecurity http) throws Exception {
    http.securityMatcher("/api/**")
        .authorizeHttpRequests(authorize -> authorize
            .requestMatchers(HttpMethod.GET, "/api/books/**").permitAll()
            .requestMatchers(HttpMethod.POST, "/api/books/**")
                .hasAnyAuthority("ROLE_ADMIN", "SCOPE_books.write") // a person
                ↪ or a token
            .requestMatchers("/api/me", "/api/orders/**").authenticated()
            .anyRequest().denyAll() // deny what no rule
                ↪ allows
        .httpBasic(withDefaults()) // username +
            ↪ password (curl, scripts) ...
        .oauth2ResourceServer(resourceServer -> resourceServer.jwt(withDefaults()))
            ↪ // ... or a JWT
        .sessionManagement(session ->
            ↪ session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        // No session cookie. Careful: browsers cache Basic credentials and send them
        ↪ on their own -
        // for a browser front end use bearer tokens only, or keep CSRF on (lesson 3.3)
        .csrf(csrf -> csrf.disable())
        .cors(withDefaults()); // uses the
            ↪ CorsConfigurationSource bean
    return http.build();
}
```

- securityMatcher("/api/\*\*"): Bu zincir yalnızca API isteklerini işler. Sıra numarası tarayıcı zincirinden küçüktür, bu yüzden önce ona sorulur.
- Kitapları okumak herkese açıktır. Yazmak ROLE\_ADMIN (bir kişi) **veya** SCOPE\_books.write (bir token) ister.
- Kimlik bilgisi yok → WWW-Authenticate header'lı **401**. Bir müşterinin geçerli kimlik bilgileri → **403**.

- STATELESS: Oturum oluşturulmaz, her istek kendi kimliğini doğrulamalıdır. CSRF'nin burada kapatılabilmesinin nedeni budur.
- `anyRequest().denyAll()`: Hiçbir kuralın bahsetmediği bir API yolu, admin için bile reddedilir. Yeni endpoint'ler biri onlar için kural yazana kadar kapalı kalır.

#### ! Uyarı

CSRF'yi kapatmak **bearer token**'lar için güvenlidir, çünkü tarayıcı onları asla kendiliğinden eklemeyi. HTTP Basic farklıdır: Kullanıcı Basic kimlik bilgilerini tarayıcının penceresine bir kez yazdıktan sonra tarayıcı onları sonraki isteklerde kendiliğinden gönderir. Bu zincirdeki HTTP Basic curl ve script'ler içindir. Bir tarayıcı ön yüzü için API'de yalnızca bearer token kabul edin veya CSRF korumasını açık tutun.

## 3.4 Method Security

URL kuralları "bir müşteri yalnızca **kendi** siparişini görebilir" kuralını ifade edemez. Method security kuralı verinin olduğu yerde kontrol eder:

```
@Service
public class OrderService {

    private final Map<Long, Order> orders = Map.of(
        1L, new Order(1, "ada", "9780134685991", 1),
        2L, new Order(2, "bob", "9781617297571", 2));

    @PreAuthorize("hasRole('ADMIN')") // checked before
    // the method runs
    public List<Order> findAll() {
        return List.copyOf(orders.values());
    }

    @PostAuthorize("returnObject.customer() == authentication.name or hasRole('ADMIN')")
    // checked on the result
    public Order find(long id) {
        Order order = orders.get(id);
        if (order == null) {
            throw new NoSuchElementException("No order " + id);
        }
        return order;
    }
}
```

- `@PreAuthorize` metottan önce çalışır: `findAll` yalnızca admin'ler içindir.
- `@PostAuthorize` metottan sonra çalışır ve sonuca (`returnObject`) bakabilir: Ada 1 numaralı siparişi alır, ancak Bob'un 2 numaralı siparişi için bir istek **403** ile biter.

- Kurallar yalnızca bir URL için değil, servisin her çağırımı için geçerlidir. `OrderSecurityTest`, `findAll()`'u doğrudan çağırır ve bir `AuthorizationDeniedException` alır.
- `@PostAuthorize` metottan **sonra** çalışır. Bu yüzden onu yalnızca yan etkisi olmayan metotlarda (okumalarda) kullanın.
- Bilinmeyen bir id **404** (controller'dan bir `ProblemDetail`), başkasının siparişi **403** verir. Bu fark, çağırana hangi id'lerin var olduğunu söyler. Bunun önemli olduğu yerde başkasının siparişleri için de 404 döndürün.
- Kural siparişin sahibini `authentication.name` ile karşılaştırır. `Client-credentials` token'ında ad client id'sidir ve bazı kimlik sağlayıcılar kullanıcıların görünen adlarını değiştirmesine izin verir. Üretimde değişebilen bir adla değil, sabit ve benzersiz bir id ile (ör. sub claim'i) karşılaştırın.

### 3.5 JWT Resource Server

API zincirindeki `oauth2ResourceServer(jwt)`, `Authorization: Bearer <token>` kabul eder. Resource server her istekte:

1. imzayı token'ı üretenin public key'i ile kontrol eder,
2. süreyi (exp) kontrol eder,
3. scope claim'ini `SCOPE_...` yetkilerine çevirir.

Sunucuda hiçbir şey saklanmaz: Kimliği token'ın kendisi taşır. `TokenFlowTest` imzanın dört karakterini değiştirir ve istek **401** ile reddedilir.

Spring Security 7 ayrıca kullanıcının kimliğini nasıl doğruladığını kaydeden bir **faktör yetkisi** ekler: token için `FACTOR_BEARER`, parola için `FACTOR_PASSWORD` (bkz. `/api/me` çıktısı). Kurallar belirli faktörler isteyebilir. Çok faktörlü kimlik doğrulamanın temeli budur.

### 3.6 Kendi Authorization Server'ımız

Token'lar aynı uygulamada çalışan Spring Authorization Server'dan gelir. `Client`, `application.yml` içinde kayıtlıdır:

```
security:
  oauth2:
    authorizationserver:
      client:
        bookstore-cli: # a machine client (e.g. a script or
↪ another service)
        registration:
          client-id: bookstore-cli
          # No default: without BOOKSTORE_CLI_SECRET the application does not start
↪ (fail fast).
          # Store it hashed, e.g. BOOKSTORE_CLI_SECRET={bcrypt}$2a$10$... – the dev
↪ profile sets a demo value.
```

```
client-secret: ${BOOKSTORE_CLI_SECRET}
client-authentication-methods: client_secret_basic
authorization-grant-types: client_credentials
scopes: books.read,books.write
```

İlk filter chain yalnızca Authorization Server endpoint'lerini işler:

```
@Configuration(proxyBeanMethods = false)
public class AuthorizationServerConfiguration {

    static final String CLIENT_SECRET =
        ↪ "spring.security.oauth2.authorizationserver.client.bookstore-
        ↪ cli.registration.client-secret";

    AuthorizationServerConfiguration(Environment environment) {
        environment.getRequiredProperty(CLIENT_SECRET); // fail fast if
        ↪ BOOKSTORE_CLI_SECRET is missing
    }

    @Bean
    @Order(1)
    SecurityFilterChain authorizationServerChain(HttpSecurity http) throws Exception {
        var authorizationServer = new OAuth2AuthorizationServerConfigurer();
        http.securityMatcher(authorizationServer.getEndpointsMatcher()) // only
        ↪ these URLs
            .with(authorizationServer, Customizer.withDefaults())
            .authorizeHttpRequests(authorize ->
                ↪ authorize.anyRequest().authenticated());
        return http.build();
    }
}
```

Gerisini Boot ekler: bir RSA imzalama anahtarı, /oauth2/jwks altında public key'ler, /.well-known/oauth-authorization-server altında metadata ve resource server'ın kullandığı bir JwtDecoder.

```
curl -u bookstore-cli:dev-only-secret -d grant_type=client_credentials -d
    ↪ scope=books.read \
    http://localhost:8080/oauth2/token
```

#### **i Not**

client\_credentials makineler içindir (bir script, başka bir servis). İnsan kullanıcıli uygulamalar PKCE ile authorization\_code grant'ini kullanır: Kullanıcı Authorization Server'da giriş yapar ve uygulama parolayı hiç görmez.

**! Önemli**

Boot her açılışta yeni bir imzalama anahtarı üretir. Yeniden başlatmadan önce üretilen token'lar geçersiz olur. Gerçek bir Authorization Server anahtarını bir key store'dan yükler ve yeniden başlatmalar arasında korur.

### 3.7 OAuth2 Login (“GitHub ile giriş yap”)

OAuth2 login’de uygulama, harici bir sağlayıcının **client**’ıdır. Yalnızca bir sağlayıcı yapılandırılmışsa açılır (bkz. bölüm 3.1’deki tarayıcı zinciri). GitHub için:

```
spring:
  security:
    oauth2:
      client:
        registration:
          github:
            client-id: ${GITHUB_CLIENT_ID}
            client-secret: ${GITHUB_CLIENT_SECRET}
```

Spring Security GitHub’ın URL’lerini bilir. Giriş sayfası bu durumda <https://github.com/login/oauth/authorize> adresine yönlendiren bir “GitHub” bağlantısı gösterir. OAuth2LoginTest yönlendirmeyi kontrol eder ve geri dönen bir kullanıcıyı GitHub’ı çağırmadan simüle etmek için spring-security-test’teki `oauth2Login()`’i kullanır.

### 3.8 CORS

<http://localhost:5173> üzerindeki bir ön yüz (ör. bir Vite geliştirme sunucusu) API’yi çağırabilir. Başka hiçbir origin çağıramaz:

```
@Bean
CorsConfigurationSource
↪ corsConfigurationSource(@Value("${bookstore.cors.allowed-origins}") List<String>
↪ origins) {
    var api = new CorsConfiguration();
    api.setAllowedOrigins(origins); // never "*" together with
    ↪ credentials
    api.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE"));
    api.setAllowedHeaders(List.of("Authorization", "Content-Type"));
    api.setMaxAge(3600L); // browsers may cache the
    ↪ preflight for an hour
    var source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/api/**", api);
    return source;
```

}

API zincirindeki `.cors(withDefaults())` bu bean'i kullanır. `http://localhost:5173`'ten gelen bir preflight `Access-Control-Allow-Origin` alır, başka bir origin'den gelen preflight **403** alır.

### 3.9 Test

`spring-security-test`, bir testte kurulması zor olan parçaları taklit eder:

| Yardımcı                                                      | Simüle ettiği                                    |
|---------------------------------------------------------------|--------------------------------------------------|
| <code>user("ada").roles("CUSTOMER"),<br/>@WithMockUser</code> | giriş yapmış bir kullanıcı                       |
| <code>httpBasic("ada", "ada-password")</code>                 | gerçek kullanıcılara karşı gerçek HTTP Basic     |
| <code>formLogin().user(...).password(...)</code>              | gerçek bir form login                            |
| <code>jwt().authorities(...)</code>                           | geçerli bir JWT'li istek                         |
| <code>oauth2Login()</code>                                    | bir OAuth2 sağlayıcısıyla giriş yapmış kullanıcı |
| <code>csrf()</code>                                           | geçerli bir CSRF token'ı                         |

`TokenFlowTest` bir adım öteye gider ve `Authorization Server`'dan gerçek bir token alır.

## 4. Sık Yapılan Hatalar ve En İyi Pratikler

### !! Dikkat

Parolaları asla düz metin olarak veya hızlı bir hash ile (MD5, SHA-256) saklamayın. `DelegatingPasswordEncoder` üzerinden `BCrypt`, `SCrypt` veya `Argon2` kullanın.

- **Yapın:** Varsayılan olarak reddedin (`anyRequest().authenticated()`) ve yalnızca herkese açık olması gerekeni açın.
- **Yapmayın:** Oturum cookie'si kullanan bir zincirde CSRF'yi kapatmayın. Yalnızca stateless bir zincir onusuz olabilir.
- **Yapın:** Sahiplik kurallarını `method security` ile servise koyun. Aksi hâlde ikinci bir controller veya bir batch işi onları atlar.
- **Yapmayın:** Kimlik bilgileriyle birlikte `setAllowedOrigins("*")` kullanmayın ve gerektenden fazla metod ve header'a izin vermeyin.

- **Yapın:** Access token'ları kısa ömürlü tutun ve token'lar harici bir sağlayıcıdan geliyorsa issuer ve audience'ı doğrulayın.
- **Yapmayın:** Secret'ları commit etmeyin. BOOKSTORE\_CLI\_SECRET placeholder'ının gösterdiği gibi ortam değişkenleri veya bir secret deposu kullanın.
- **Yapın:** Hiçbir kuralın izin vermediğini reddedin (API'de `anyRequest().denyAll()`).
- **Yapmayın:** Bilinen parolalı demo kullanıcıları her ortamda yüklemeyin. Onları yalnızca geliştirme için olan bir profile ve migration konumunda tutun.

## 5. Özet

- Kimlik doğrulama “kim?” (401), yetkilendirme “ne?” (403) sorusunu yanıtlar. İkisini de controller'ınız çalışmadan önce filter chain yapar.
- Eşleyicileri ve sıraları olan birden çok `SecurityFilterChain`, API'yi tarayıcıdan ayırır.
- Kullanıcılar PostgreSQL'de durabilir. Parolalar, bir algoritma öneki arkasında tuzlanmış ve yavaş BCrypt hash'leri olarak saklanır.
- Tarayıcı uygulamaları CSRF korumasına ihtiyaç duyar, stateless API'ler duymaz. CORS, hangi başka origin'lerin API'yi çağırabileceğini belirler.
- `@PreAuthorize` ve `@PostAuthorize`, “yalnızca kendi siparişlerin” gibi kuralları serviste uygular.
- Spring Authorization Server imzalı JWT'ler üretir, resource server her istekte imzayı, süreyi ve scope'ları kontrol eder.
- `spring-security-test`, testlerde kullanıcıları, token'ları ve girişleri simüle eder.

Sıradaki adım: [Ödevler](#)

## 6. İleri Okuma

- [Spring Security Reference](#) · [Servlet Architecture](#)
- [Password Storage](#) · [CSRF](#) · [CORS](#)
- [Method Security](#)
- [OAuth 2.0 Resource Server — JWT](#) · [OAuth2 Login](#)
- [Spring Authorization Server](#)
- [Spring Boot — Security](#)
- [OWASP Password Storage Cheat Sheet](#) · [Authentication Cheat Sheet](#) (brute-force koruması, hesap kilitleme)